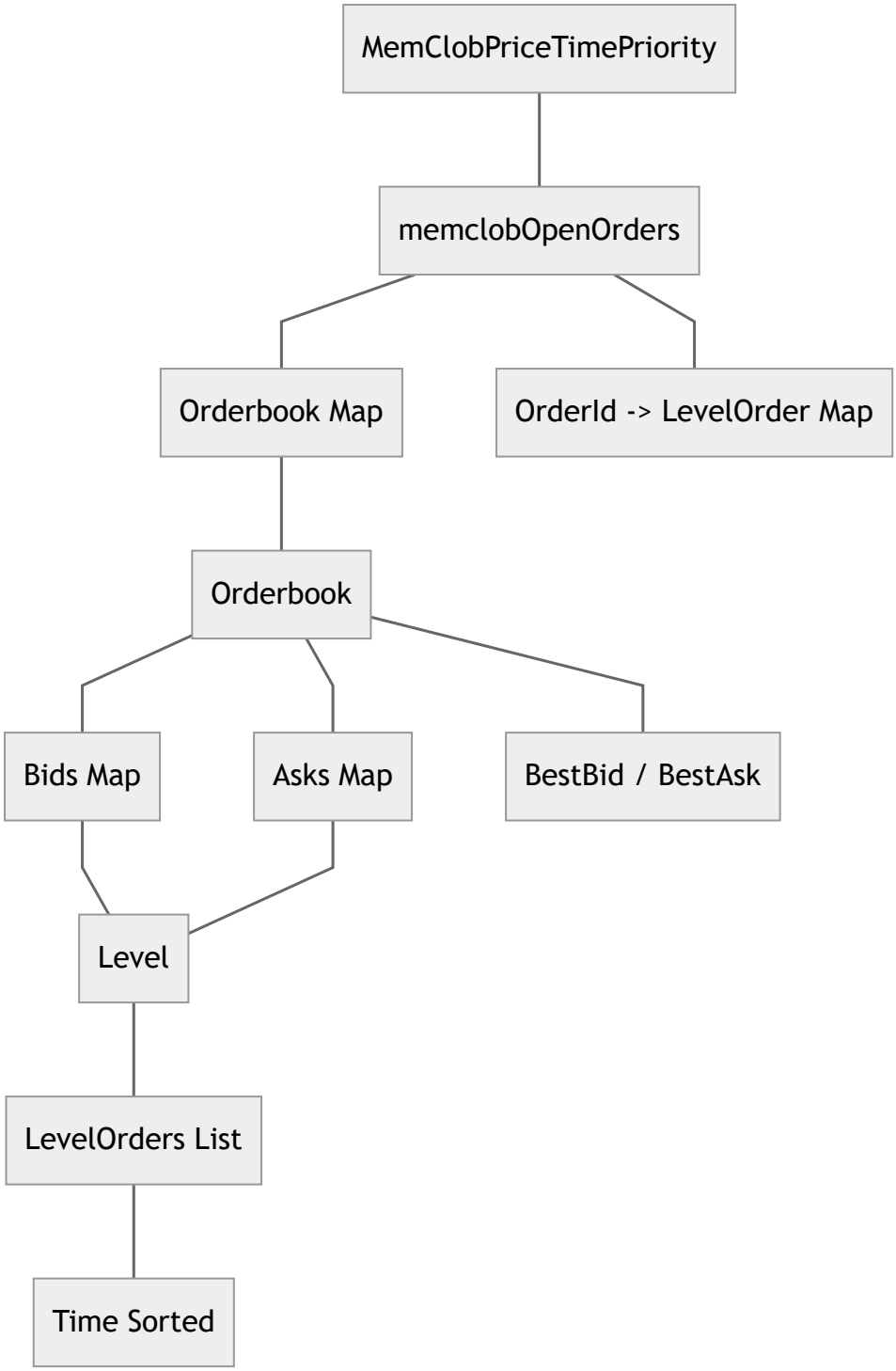
dYdX v4 거래 처리 플로우 - 매칭 엔진 핵심 알고리즘 분석

개요

이 문서는 dYdX v4의 매칭 엔진 핵심 알고리즘을 상세히 분석합니다. 주문서 관리 및 매칭 알고리즘의 구현 세부사항, 가격 레벨별 주문 정렬 및 매칭 로직, 그리고 시장가 주문 및 지정가 주문 처리 방식을 포함합니다.

매칭 엔진 아키텍처

전체 구조



주문서 관리 및 매칭 알고리즘 상세 구현 분석

1. 주문서 데이터 구조

Orderbook 구조

```
type Orderbook struct {
```

```
// 틱 크기 정의 (서브틱 단위)
SubticksPerTick SubticksPerTick
// 가격별 매수 주문 레벨
Bids map[Subticks]*Level
// 가격별 매도 주문 레벨
Asks map[Subticks]*Level
// 최고 매수 가격 (0이면 매수 주문 없음)
BestBid Subticks
// 최저 매도 가격 (MaxUint64면 매도 주문 없음)
BestAsk Subticks
// 서브계정별 열린 주문 추적
SubaccountOpenClobOrders map[satypes.SubaccountId]map[Order_Side]map[OrderId]boo
// 최소 주문 크기
MinOrderBaseQuantums satypes.BaseQuantums
// 서브계정별 Reduce-Only 주문 추적
SubaccountOpenReduceOnlyOrders map[satypes.SubaccountId]map[OrderId]bool
// 총 열린 주문 수 (관찰용)
TotalOpenOrders uint
}
```

Level 구조 (가격 레벨)

```
type Level struct {
```

```
// 시간 순서로 정렬된 주문들의 이중 연결 리스트

// 먼저 들어온 주문이 앞에 위치 (FIFO)

LevelOrders list.List[ClobOrder]

}


type ClobOrder struct {

// 실제 주문 정보

Order Order

// 트랜잭션 서명 (ABCI++ 대응)

Signature []byte

}
```

2. 매칭 엔진 핵심 로직

주문 매칭 메인 함수

```
func (m *MemClobPriceTimePriority) matchOrder(

ctx sdk.Context,

order types.MatchableOrder,

) (

orderStatus types.TakerOrderStatus,

offchainUpdates *types.OffchainUpdates,

makerOrdersToRemove []OrderWithRemovalReason,

err error,
```

```
) {
offchainUpdates = types.NewOffchainUpdates()
// 상태 분기 - 매칭 실패 시 롤백을 위해
branchedContext, writeCache := ctx.CacheContext()
// 주문서에 대해 매칭 시도
newMakerFills,
matchedOrderHashToOrder,
matchedMakerOrderIdToOrder,
makerOrdersToRemove,
takerOrderStatus := m.mustPerformTakerOrderMatching(
branchedContext,
order,
)
// 교체 주문인 경우 기존 주문 제거
if !order.IsLiquidation() {
orderId := order.MustGetOrder().OrderId
if orderToBeReplaced, found := m.openOrders.getOrder(branchedContext, orderId);
makerOrdersToRemove = append(makerOrdersToRemove, OrderWithRemovalReason{Order:
}
}
// 제거할 Maker 주문들 처리
for _, makerOrderWithRemovalReason := range makerOrdersToRemove {
makerOrderId := makerOrderWithRemovalReason.Order.OrderId
```

```
// 오프체인 업데이트 생성
if m.generateOffchainUpdates && (order.IsLiquidation() || makerOrderId != order.
reason := indexershared.ConvertOrderRemovalReasonToIndexerOrderRemovalReason(
makerOrderWithRemovalReason.RemovalReason,
)
if message, success := off_chain_updates.CreateOrderRemoveMessageWithReason(
branchedContext,
makerOrderId,
reason,
ocutypes.OrderRemoveV1_ORDER_REMOVAL_STATUS_BEST_EFFORT_CANCELED,
); success {
offchainUpdates.AddRemoveMessage(makerOrderId, message)
}
}
// 주문서에서 제거
m.openOrders.mustRemoveOrder(branchedContext, &makerOrderWithRemovalReason.Order
}
// 매칭 성공 시 상태 커밋
writeCache()
return takerOrderStatus, offchainUpdates, makerOrdersToRemove, nil
}
```

Taker 주문 매칭 수행

```go
func (m *MemClobPriceTimePriority) mustPerformTakerOrderMatching(
ctx sdk.Context,
takerOrder types.MatchableOrder,
) (
newMakerFills []types.MakerFill,
matchedOrderHashToOrder map[types.OrderHash]types.ClobOrder,
matchedMakerOrderIdToOrder map[types.OrderId]types.Order,
makerOrdersToRemove []OrderWithRemovalReason,
takerOrderStatus types.TakerOrderStatus,
) {
// 초기화
newMakerFills = []types.MakerFill{}
matchedOrderHashToOrder = make(map[types.OrderHash]types.ClobOrder)
matchedMakerOrderIdToOrder = make(map[types.OrderId]types.Order)
makerOrdersToRemove = []OrderWithRemovalReason{}
// Taker 주문 상태 초기화
takerOrderStatus = types.TakerOrderStatus{
OrderStatus: types.Success,
RemainingQuantums: takerOrder.GetBaseQuantums(),
OrderOptimisticallyFilledQuantums: 0,
}
// 주문서 가져오기
```

```
orderbook := m.openOrders.mustGetOrderbook(ctx, takerOrder.GetClobPairId())

// 반대편 주문들과 매칭 시도

oppositeSide := orderbook.GetSide(!takerOrder.IsBuy())

// 가격 순서대로 매칭 시도

for {

// 최적 가격의 주문 찾기

bestLevelOrder, found := m.openOrders.getBestOrderOnSide(orderbook, !takerOrder.

if !found {

break // 매칭할 주문이 없음

}

makerOrder := bestLevelOrder.Value.Order

// 가격 매칭 조건 확인

if !m.canMatchOrders(takerOrder, makerOrder) {

break // 더 이상 매칭 불가능

}

// 매칭 수량 계산

matchQuantums := min(takerOrderStatus.RemainingQuantums, makerOrder.GetBaseQuant

// 담보 확인

if !m.checkCollateralization(ctx, takerOrder, makerOrder, matchQuantums) {

// 담보 부족 시 Maker 주문 제거

makerOrdersToRemove = append(makerOrdersToRemove, OrderWithRemovalReason{

Order: bestLevelOrder.Value,

RemovalReason: types.OrderRemoval_REMOVAL_REASON_UNDERCOLLATERALIZED,
```

```
})
continue
}
// 매칭 실행
fill := types.MakerFill{
MakerOrderId: makerOrder.OrderId,
FillAmount: matchQuantums,
}
newMakerFills = append(newMakerFills, fill)
// 상태 업데이트
takerOrderStatus.RemainingQuantums -= matchQuantums
takerOrderStatus.OrderOptimisticallyFilledQuantums += matchQuantums
// Maker 주문 처리
if makerOrder.GetBaseQuantums() == matchQuantums {
// 완전 체결 - 제거
makerOrdersToRemove = append(makerOrdersToRemove, OrderWithRemovalReason{
Order: bestLevelOrder.Value,
RemovalReason: types.OrderRemoval_REMOVAL_REASON_FULLY_FILLED,
})
} else {
// 부분 체결 - 수량 업데이트
m.updateOrderQuantums(ctx, bestLevelOrder, makerOrder.GetBaseQuantums()-matchQua
}
```

```
// 매칭 기록

matchedOrderHashToOrder[makerOrder.GetOrderHash()] = bestLevelOrder.Value

matchedMakerOrderIdToOrder[makerOrder.OrderId] = makerOrder

// Taker 주문이 완전 체결되면 종료

if takerOrderStatus.RemainingQuantums == 0 {

break

}

}

return newMakerFills, matchedOrderHashToOrder, matchedMakerOrderIdToOrder, maker

}
```

가격 레벨별 주문 정렬 및 매칭 로직

1. 가격 레벨 관리

최적 주문 찾기

```
func (m *memclobOpenOrders) getBestOrderOnSide(

orderbook *types.Orderbook,

isBuy bool,

) (

bestOrder *types.LevelOrder,

found bool,

) {
```

```
var bestPrice types.Subticks
if isBuy {
// 매수: 최고 가격 (BestBid)
bestPrice = orderbook.BestBid
if bestPrice == 0 {
return nil, false // 매수 주문 없음
}
} else {
// 매도: 최저 가격 (BestAsk)
bestPrice = orderbook.BestAsk
if bestPrice == math.MaxUint64 {
return nil, false // 매도 주문 없음
}
}
// 해당 가격 레벨에서 첫 번째 주문 반환
return m.getFirstOrderAtSideAndSubticks(orderbook, isBuy, bestPrice)
}
```

특정 가격 레벨의 첫 번째 주문 가져오기

```
func (m *memclobOpenOrders) getFirstOrderAtSideAndSubticks(
orderbook *types.Orderbook,
isBuy bool,
```

```
subticks types.Subticks,
) (
firstOrder *types.LevelOrder,
found bool,
) {
// 해당 사이드의 레벨 맵 가져오기
levels := orderbook.GetSide(isBuy)
// 특정 가격 레벨 찾기
level, exists := levels[subticks]
if !exists || level.LevelOrders.Len() == 0 {
return nil, false
}
// 시간 순서상 첫 번째 주문 반환 (FIFO)
return level.LevelOrders.Front(), true
}
```

2. 주문 추가 및 제거

주문서에 주문 추가

```
func (m *memclobOpenOrders) mustAddOrderToOrderbook(
ctx sdk.Context,
newOrder types.Order,
) {
```

```
clobPairId := newOrder.GetClobPairId()

orderbook := m.mustGetOrderbook(ctx, clobPairId)

// 해당 사이드의 레벨 맵 가져오기

orders := orderbook.GetSide(newOrder.IsBuy())

// 가격 레벨 가져오기 또는 생성

level, exists := orders[newOrder.GetOrderSubticks()]

if !exists {

level = &types.Level{

LevelOrders: *list.New[types.ClobOrder](),

}

orders[newOrder.GetOrderSubticks()] = level

}

// 주문을 레벨 끝에 추가 (시간 순서 유지)

clobOrder := types.ClobOrder{

Order: newOrder,

Signature: []byte{}, // TODO: 실제 서명 추가

}

levelOrder := level.LevelOrders.PushBack(clobOrder)

// 빠른 검색을 위한 맵 업데이트

m.orderIdToLevelOrder[newOrder.OrderId] = levelOrder

// BestBid/BestAsk 업데이트

m.updateBestPrices(orderbook, newOrder.GetOrderSubticks(), newOrder.IsBuy(), tru

// 서브계정 주문 추적 업데이트
```

```
m.updateSubaccountOrders(orderbook, newOrder, true)

// 총 주문 수 증가

orderbook.TotalOpenOrders++

}
```

주문서에서 주문 제거

```
func (m *memclobOpenOrders) mustRemoveOrder(

ctx sdk.Context,

levelOrder *types.LevelOrder,

) {

order := levelOrder.Value.Order

clobPairId := order.GetClobPairId()

orderbook := m.mustGetOrderbook(ctx, clobPairId)

// 해당 레벨에서 주문 제거

subticks := order.GetOrderSubticks()

levels := orderbook.GetSide(order.IsBuy())

level := levels[subticks]

level.LevelOrders.Remove(levelOrder)

// 레벨이 비어있으면 제거

if level.LevelOrders.Len() == 0 {

delete(levels, subticks)

// BestBid/BestAsk 업데이트 필요

m.updateBestPricesAfterRemoval(orderbook, subticks, order.IsBuy())
```

```
}

// 맵에서 제거

delete(m.orderIdToLevelOrder, order.OrderId)

// 서브계정 주문 추적 업데이트

m.updateSubaccountOrders(orderbook, order, false)

// 총 주문 수 감소

orderbook.TotalOpenOrders--

}
```

3. BestBid/BestAsk 관리

최적 가격 업데이트

```
func (m *memclobOpenOrders) updateBestPrices(

orderbook *types.Orderbook,

newSubticks types.Subticks,

isBuy bool,

isAddition bool,

) {

if isBuy {

// 매수 주문: 더 높은 가격이 더 좋음

if isAddition {

if newSubticks > orderbook.BestBid {

orderbook.BestBid = newSubticks
```

```
}
}
// 제거 시에는 별도 함수에서 처리
} else {
// 매도 주문: 더 낮은 가격이 더 좋음
if isAddition {
if newSubticks < orderbook.BestAsk {
orderbook.BestAsk = newSubticks
}
}
// 제거 시에는 별도 함수에서 처리
}
}


func (m *memclobOpenOrders) updateBestPricesAfterRemoval(
orderbook *types.Orderbook,
removedSubticks types.Subticks,
isBuy bool,
) {
if isBuy && removedSubticks == orderbook.BestBid {
// 최고 매수 가격이 제거됨 - 다음 최고 가격 찾기
orderbook.BestBid = m.findNextBestBid(orderbook)
} else if !isBuy && removedSubticks == orderbook.BestAsk {
```

```
// 최저 매도 가격이 제거됨 - 다음 최저 가격 찾기
orderbook.BestAsk = m.findNextBestAsk(orderbook)
}
}
```

시장가 주문 및 지정가 주문 처리 방식 분석

1. 주문 타입별 매칭 조건

매칭 가능 여부 확인

```
func (m *MemClobPriceTimePriority) canMatchOrders(
takerOrder types.MatchableOrder,
makerOrder types.Order,
) bool {
takerPrice := takerOrder.GetOrderSubticks()
makerPrice := makerOrder.GetOrderSubticks()
if takerOrder.IsBuy() {
// 매수 Taker: Taker 가격이 Maker 가격보다 높거나 같아야 매칭
// (매수자가 더 높은 가격을 제시하면 매칭 가능)
return takerPrice >= makerPrice
} else {
// 매도 Taker: Taker 가격이 Maker 가격보다 낮거나 같아야 매칭
// (매도자가 더 낮은 가격을 제시하면 매칭 가능)
```

```
return takerPrice <= makerPrice
}
}
```

2. 지정가 주문 (Limit Order) 처리

지정가 주문 특징

- **가격 제한**: 지정된 가격 이상/이하에서만 체결
- **부분 체결 허용**: 일부만 체결되고 나머지는 주문서에 대기
- **시간 우선순위**: 같은 가격에서는 먼저 들어온 주문이 우선

지정가 주문 매칭 로직

```
func (m *MemClobPriceTimePriority) processLimitOrder(
ctx sdk.Context,
order types.Order,
) (
orderStatus types.TakerOrderStatus,
offchainUpdates *types.OffchainUpdates,
err error,
) {
// 1. 매칭 시도
takerOrderStatus, offchainUpdates, _, err := m.matchOrder(ctx, order)
if err != nil {
```

```
return takerOrderStatus, offchainUpdates, err

}

// 2. 남은 수량이 있으면 주문서에 추가

if takerOrderStatus.RemainingQuantums > 0 {

remainingOrder := types.Order{

OrderId: order.OrderId,

Side: order.Side,

Quantums: takerOrderStatus.RemainingQuantums,

Subticks: order.Subticks,

TimeInForce: order.TimeInForce,

ReduceOnly: order.ReduceOnly,

ClientId: order.ClientId,

}

// 담보 확인 후 주문서에 추가

if m.checkCollateralizationForAddToOrderbook(ctx, remainingOrder) {

m.openOrders.mustAddOrderToOrderbook(ctx, remainingOrder)

// 주문서 추가 이벤트 생성

if m.generateOffchainUpdates {

if message, success := off_chain_updates.CreateOrderPlaceMessage(

ctx,

remainingOrder,

); success {

offchainUpdates.AddPlaceMessage(remainingOrder.OrderId, message)
```

```
}

}

} else {

takerOrderStatus.OrderStatus = types.Undercollateralized

}

}

return takerOrderStatus, offchainUpdates, nil

}
```

3. 시장가 주문 (Market Order) 처리

시장가 주문 특징

- **즉시 체결**: 현재 주문서의 최적 가격으로 즉시 체결
- **가격 제한 없음**: 시장에서 가능한 모든 가격에서 체결
- **주문서에 대기하지 않음**: 체결되지 않은 부분은 취소

시장가 주문 매칭 로직

```
func (m *MemClobPriceTimePriority) processMarketOrder(

ctx sdk.Context,

order types.Order,

) (

orderStatus types.TakerOrderStatus,

offchainUpdates *types.OffchainUpdates,

err error,
```

```
) {
// 시장가 주문은 매우 높은/낮은 가격으로 설정하여 모든 레벨과 매칭되도록 함
marketOrder := order
if order.IsBuy() {
// 매수 시장가: 매우 높은 가격 설정
marketOrder.Subticks = math.MaxUint64
} else {
// 매도 시장가: 매우 낮은 가격 설정 (0)
marketOrder.Subticks = 0
}
// 매칭 시도 (주문서에 추가하지 않음)
takerOrderStatus, offchainUpdates, _, err := m.matchOrder(ctx, marketOrder)
if err != nil {
return takerOrderStatus, offchainUpdates, err
}
// 시장가 주문은 남은 수량이 있어도 주문서에 추가하지 않음
if takerOrderStatus.RemainingQuantums > 0 {
// 부분 체결로 처리 (나머지는 취소됨)
takerOrderStatus.OrderStatus = types.Success // 또는 별도 상태
}
return takerOrderStatus, offchainUpdates, nil
}
```

4. IOC (Immediate-Or-Cancel) 주문 처리

IOC 주문 특징

- **즉시 체결 또는 취소**: 즉시 체결되지 않는 부분은 취소
- **주문서에 대기하지 않음**: 지정가 주문이지만 주문서에 추가되지 않음

IOC 주문 처리 로직

```
func (m *MemClobPriceTimePriority) processIOCOrder(
ctx sdk.Context,
order types.Order,
) (
orderStatus types.TakerOrderStatus,
offchainUpdates *types.OffchainUpdates,
err error,
) {
// 매칭 시도
takerOrderStatus, offchainUpdates, _, err := m.matchOrder(ctx, order)
if err != nil {
return takerOrderStatus, offchainUpdates, err
}
// IOC 주문은 남은 수량이 있어도 주문서에 추가하지 않음
if takerOrderStatus.RemainingQuantums > 0 {
takerOrderStatus.OrderStatus = types.ImmediateOrCancelWouldRestOnBook
}
```

```
return takerOrderStatus, offchainUpdates, nil
}
```

매칭 성능 최적화

1. 데이터 구조 최적화

이중 연결 리스트 사용

```
// 시간 복잡도 최적화
// - 주문 추가: O(1) - 리스트 끝에 추가
// - 주문 제거: O(1) - 직접 참조로 제거
// - 첫 번째 주문 접근: O(1) - 리스트 헤드 접근
type Level struct {
LevelOrders list.List[ClobOrder] // 이중 연결 리스트
}
```

해시맵 기반 빠른 검색

```
type memclobOpenOrders struct {
// O(1) 주문서 접근
orderbooksMap map[types.ClobPairId]*types.Orderbook
// O(1) 주문 검색 및 제거
orderIdToLevelOrder map[types.OrderId]*types.LevelOrder
```

```
// O(1) 만료 주문 관리

blockExpirationsForOrders map[uint32]map[types.OrderId]bool

}
```

2. 메모리 관리 최적화

객체 재사용

```
// 자주 생성/삭제되는 객체의 풀링

var (

levelPool = sync.Pool{

New: func() interface{} {

return &types.Level{

LevelOrders: *list.New[types.ClobOrder](),

}

},

}

)


func (m *memclobOpenOrders) getLevel() *types.Level {

return levelPool.Get().(*types.Level)

}


func (m *memclobOpenOrders) putLevel(level *types.Level) {
```

```
level.LevelOrders.Clear()

levelPool.Put(level)

}
```

3. 알고리즘 최적화

조기 종료 조건

```
func (m *MemClobPriceTimePriority) mustPerformTakerOrderMatching(

ctx sdk.Context,

takerOrder types.MatchableOrder,

) {

// 매칭 불가능한 조건에서 조기 종료

for {

bestLevelOrder, found := m.openOrders.getBestOrderOnSide(orderbook, !takerOrder.

if !found {

break // 반대편 주문 없음

}

makerOrder := bestLevelOrder.Value.Order

// 가격 조건 불만족 시 즉시 종료

if !m.canMatchOrders(takerOrder, makerOrder) {

break

}

// 매칭 로직...
```

```
  }

}
```

결론

dYdX v4의 매칭 엔진은 고성능 탈중앙화 거래소의 요구사항을 충족하기 위해 정교하게 설계된 알고리즘을 사용합니다. 가격-시간 우선순위 기반의 매칭, 효율적인 데이터 구조 활용, 그리고 다양한 주문 타입 지원을 통해 기존 중앙화 거래소에 필적하는 성능을 제공합니다. 특히 이중 연결 리스트와 해시맵을 조합한 데이터 구조는 O(1) 시간 복잡도로 주요 연산을 수행할 수 있게 하여 높은 처리량을 달성합니다.